Overview

Post-training is the stage where a pretrained language model is adapted for a concrete product context. In practice, this means improving instruction following, output format consistency, domain language behavior, and safety style while controlling compute cost.

A useful mental model is: 1. **Pretraining** teaches general language priors. 2. **Post-training** teaches task behavior and preference alignment. 3. **Application orchestration** (RAG, tools, agents) grounds behavior with live context.

This chapter focuses on post-training workflows used by modern AI engineering teams: supervised fine-tuning (SFT), parameter-efficient fine-tuning (PEFT), LoRA, QLoRA, and the bridge to preference optimization methods (DPO/RLHF).

Why post-training exists

Prompt engineering is fast and often enough for prototypes. But repeated prompts can fail to enforce: - strict schema outputs, - domain-specific tone/terminology, - safety style consistency, - stable behavior under input variation.

Post-training exists to convert those unstable prompt tricks into a more reliable model policy.

Post-Training Taxonomy

Supervised Fine-Tuning (SFT)

SFT trains on examples of desired input-output behavior. For each example x_i with target y_i, the objective is standard token-level next-token likelihood:

$$\mathcal{L}_{\text{SFT}} = -\sum_i \log p_\theta(y_i \mid x_i)$$

Intuition: SFT tells the model "for this prompt shape, this style of answer is correct."

Strengths: - simple training objective, - high controllability for known tasks, - straightforward evaluation against labeled sets.

Limits: - quality depends heavily on data design, - can overfit narrow styles, - does not directly optimize pairwise human preference unless preference labels are converted.

Parameter-Efficient Fine-Tuning (PEFT)

PEFT adapts a model by training a small set of extra parameters while freezing most base weights. This reduces GPU memory and speeds iteration.

Common PEFT families include: - LoRA and LoRA variants, - prefix/prompt tuning, - adapters and selective layer tuning.

In production, PEFT is popular because multiple domain adapters can be stored and switched without retraining full model weights.

LoRA (Low-Rank Adaptation)

LoRA approximates update matrices with low-rank factors. Instead of full update ΔW, learn:

$$\Delta W = BA$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$ with small rank r.

Operationally: - freeze base model, - inject trainable low-rank modules into selected layers, - train only those modules.

Trade-off knobs: - rank `r` (capacity vs memory), - target modules (where adaptation power is applied), - dropout and scaling (`alpha`) for stability.

QLoRA (Quantized LoRA)

QLoRA combines quantization and LoRA: - base model is quantized (commonly 4-bit for memory reduction), - LoRA adapters remain trainable in higher precision, - gradients flow through the frozen quantized base into adapters.

QLoRA became important because it enabled practical tuning of larger models on limited hardware. The original work introduced memory-saving mechanisms such as NF4 quantization and double quantization.

Practical implications: - lower memory footprint, - lower hardware barrier, - potential quality/throughput trade-offs compared with higher precision pipelines.

Data Design for Fine-Tuning

Post-training performance is mostly data quality, not trainer choice.

SFT dataset patterns

Common formats: - instruction -> response, - multi-turn chat with roles, - structured I/O examples (JSON schema outputs), - tool-call transcripts.

Good SFT data properties: - covers expected production prompt distribution, - includes hard negatives and boundary cases, - has consistent style policy, - avoids leakage from eval sets.

Preference data patterns

For DPO/RLHF-style tuning, you need tuples like: - prompt, - preferred response, - rejected response.

Preference labels should reflect product policy (helpfulness, safety, factuality, format adherence), not only annotator "vibe".

Data quality checklist

- Is each sample mapped to a clear business objective?
- Are there contradictory style rules across examples?
- Are safety-critical slices represented?
- Are train/validation/test splits leakage-safe?
- Do we have baseline metrics before tuning?

Training Workflow in Practice

A realistic post-training loop: 1. Define success metrics and failure slices. 2. Build evaluation harness before training. 3. Prepare versioned training data. 4. Run baseline model on eval suite. 5. Train SFT/PEFT candidate(s). 6. Compare offline metrics and human review panels. 7. Run safety and regression checks. 8. Package adapter/model with metadata. 9. Deploy behind canary or shadow traffic. 10. Monitor drift, quality, cost, and incident rate.

Choosing Prompting vs RAG vs Fine-Tuning

Use this decision heuristic: - If the issue is **missing facts**, prefer RAG/index refresh. - If the issue is **behavior/style consistency**, prefer SFT/PEFT. - If the issue is **multi-step policy preference**, consider preference optimization. - If constraints are strict and data is scarce, start with prompt + guardrails and delay tuning.

Bridge to Preference Optimization (DPO/RLHF)

SFT provides a solid behavioral baseline. Preference optimization methods refine how the model ranks competing responses.

- **RLHF** classically includes reward modeling and policy optimization.
- **DPO** directly optimizes preferred-vs-rejected behavior without explicit reward model training in the same way.

A pragmatic stack used in many teams: 1. SFT for task/style grounding, 2. DPO or related methods for preference alignment, 3. application-level guardrails for runtime safety.

Operationalization: Versioning and Rollback

Treat adapters or tuned checkpoints like release artifacts.

Minimum metadata per run: - base model ID and revision, - dataset version hash, - hyperparameters (rank, lr, epochs, sequence length), - eval suite version, - offline and human review results, - safety outcomes.

Rollback strategy: - keep previous stable adapter/model, - support instant switch via config flag, - archive incident context for postmortem and retraining queue.

Business Case Studies & Exceptions

Case 1: Domain support assistant for regulated SaaS

Problem: Base LLM gives fluent but policy-inconsistent support answers. Approach: SFT with policy-aligned support transcripts plus schema-constrained output. Outcome pattern: Better consistency and reduced escalation rate. Exception: Frequent product updates still require RAG for fresh facts.

Case 2: Internal coding assistant

Problem: Prompt-only solution produces inconsistent patch format and unsafe commands. Approach: PEFT/LoRA tuned on accepted patch patterns and secure command policies. Outcome pattern: Higher acceptance rate in code review. Exception: If repository context retrieval is weak, fine-tuning alone will not solve hallucinated file paths.

Case 3: Low-budget startup with one GPU

Problem: Need domain adaptation under tight hardware constraints. Approach: QLoRA with small high-quality dataset and strict eval gating. Outcome pattern: Major cost savings versus full fine-tuning. Exception: For very high precision domains, quantization trade-offs may require mixed precision or larger eval effort.

Interview Questions & Answers

1. **Q:** What is post-training in LLM systems?
   A: The adaptation stage after pretraining where model behavior is specialized for tasks, preferences, and product constraints.

2. **Q:** SFT objective in one sentence?
   A: Maximize likelihood of desired outputs given prompts, usually via token-level cross-entropy.
3. **Q:** Fine-tuning vs RAG?
   A: Fine-tuning changes behavior/policy; RAG injects fresh external knowledge at inference time.
4. **Q:** What is PEFT?
   A: Techniques that tune a small parameter subset rather than all model weights.
5. **Q:** Why is LoRA efficient?
   A: It constrains updates to low-rank matrices, reducing trainable parameters and memory.
6. **Q:** What does LoRA rank control?
   A: Adaptation capacity versus memory/compute cost.
7. **Q:** What is QLoRA?
   A: LoRA on top of a quantized frozen base model, typically 4-bit, to reduce memory.
8. **Q:** Main benefit of QLoRA?
   A: Enables larger-model tuning on smaller hardware budgets.
9. **Q:** Biggest risk in fine-tuning projects?
   A: Poor training/eval data quality and leakage.
10. **Q:** When is prompt engineering enough?
    A: When behavior is acceptable with low variance and product constraints are light.
11. **Q:** Why evals before tuning?
    A: Without baseline evals, you cannot prove tuning improved anything.
12. **Q:** What is a preference dataset?
    A: Prompt with at least one preferred response and one rejected response.
13. **Q:** DPO vs RLHF at high level?
    A: DPO directly optimizes preference pairs; RLHF usually uses reward modeling plus RL optimization.
14. **Q:** Can fine-tuning replace runtime guardrails?
    A: No, guardrails remain necessary for defense in depth.
15. **Q:** Why keep adapter metadata?
    A: For reproducibility, auditability, and rollback safety.
16. **Q:** What does overfitting look like in SFT?
    A: Strong in-domain style mimicry but degraded generalization on unseen prompts.
17. **Q:** How do you decide between full FT and PEFT?
    A: Use PEFT by default; consider full FT only when PEFT hits quality ceilings and budget allows.
18. **Q:** What are good post-training success metrics?
    A: Task quality, schema adherence, safety rates, latency/cost impact, and human acceptance.
19. **Q:** Why include hard negative examples?
    A: They teach boundary behavior and reduce brittle failures.
20. **Q:** One sentence production advice for tuning?
    A: Invest more in evaluation and data curation than in trainer hyperparameter tinkering.

References

- PEFT docs: https://huggingface.co/docs/peft/main/index
- LoRA paper: https://arxiv.org/abs/2106.09685
- QLoRA paper: https://arxiv.org/abs/2305.14314
- TRL docs (SFT/DPO/RL methods): https://huggingface.co/docs/trl/main/index
- DPO paper: https://arxiv.org/abs/2305.18290
- InstructGPT (RLHF pipeline): https://arxiv.org/abs/2203.02155
- OpenAI SFT guide: https://platform.openai.com/docs/guides/supervised-fine-tuning